Research Analysis: Formal Security Models in Operating Systems

Introduction

For access control, operating system security uses models. These models provide mathematical rules and conditions for access control to maintain the security of the system. Here these rules are defined like that subject, which means users or processes can interact with objects, which are files and memory. This interaction depends on the defined rules. Some of the models are (1) the Bell-LaPadula Model, (2) the Biba Model, and (3) Covert Channels in Secure Systems.

Bell-LaPadula Model: Preserving Confidentiality

Firstly, the Bell-LaPadula model, or BLP model, was named after the creators, who were David Bell and Leonard LaPadula. They created the model around the 1970s. The two main focuses of this model were

1. Protect confidentiality
2. Prevent unauthorized information disclosure.

It is used in Multi-Level Security (MLS) systems. And also include levels like Unclassified, Confidential, Secret, and Top Secret. The Bell-LaPadula model works based on the need-to-know principle. Here the information flow is only upwards in a

typical MLS hierarchy. Objects are assigned to security levels (classification), and subjects are assigned to clearance levels. This need-to-know policy prevents data leakages. This model works like “no read up” and “no write down”. For example, a person who has access to secret clearance can read secret documents, but he can't access the top-secret file and can't read it. Moreover, he can write on top-secret documents because in the BLP model writing up is allowed.

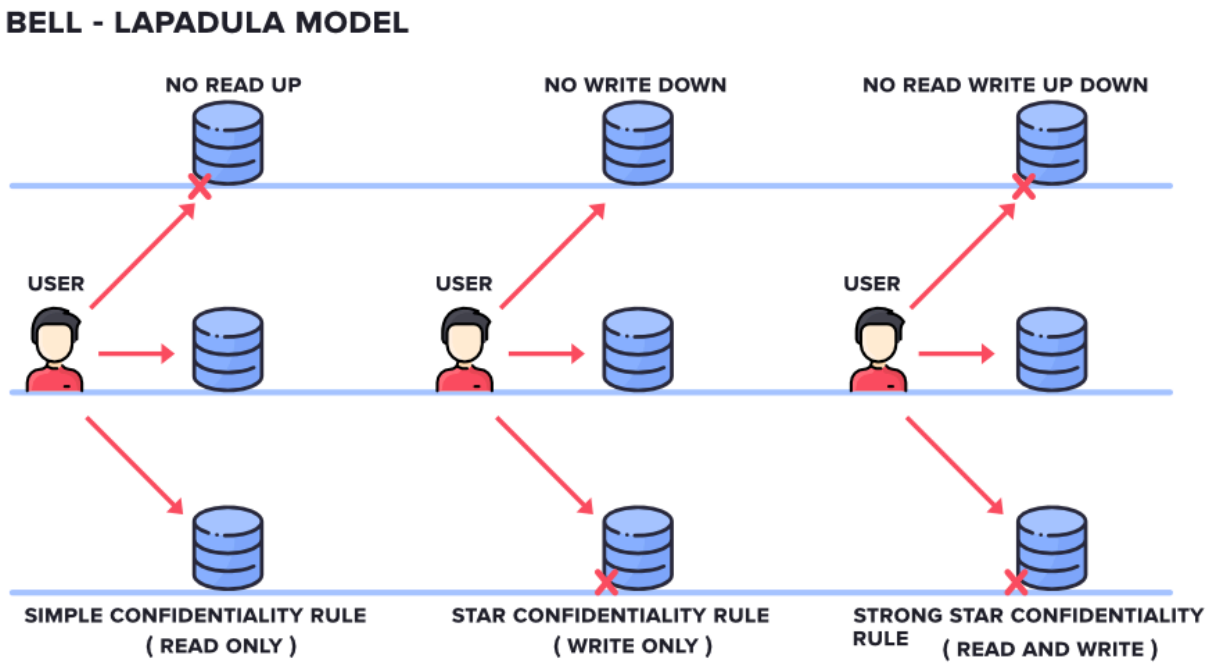

Fig. 01: Bell-LaPadula Model

There are some core properties of the Bell-LaPadula model:

1. **The Simple Security Property (No Read Up):** This protects confidentiality. Here the subject can't read any information that is at a higher level than their clearance level. Formally, for a state s, subject u, and object n:

   $(u, n) \in observe(s) \Rightarrow clearance(u) \geq classification(s, n)$ (Rushby, 1995).

2. **Star Property (No Write Down):** Here a user cannot write any information to a lower level than its own. It prevents the data from leaking to unauthorized users. For an untrusted subject u:

   $(u,n) \in alter(s) \Rightarrow classification(s,n) \geq current\text{-}level(s,u)$ (Rushby, 1995).

Suitability: If confidentiality is the most required priority, then BLP is a highly recommended model because it gives confidential protection to the system. Here formal verification is required. It strictly maintains confidentiality by preventing unauthorized access to classified information (Rushby, 1995).

For Example: Military system, Intelligence system, Government system

Limitations: The Bell-LaPadula model cannot maintain data integrity. A lower-level user may modify the upper integrity data. This limitation was the reason for developing the Biba integrity model.

Biba Model: Ensuring Data Integrity

Secondly, in 1975, the Biba model was proposed by Kenneth J. Biba. The main purpose was

1. Protect data integrity
2. Prevent unauthorized data modifications

The Biba model is different from the Bell-LaPadula model. The Bell-LaPadula model ensures confidentiality, and on the other hand, the Biba model protects the data's integrity. It checks whether the subject, meaning the user, can corrupt the data or not. It uses its integrity level like Crucial, Important, and Low Integrity. A subjects cannot read lower-level or write to higher-level objects; that's how the Biba model prevent untrusted sources from corrupting reliable data.

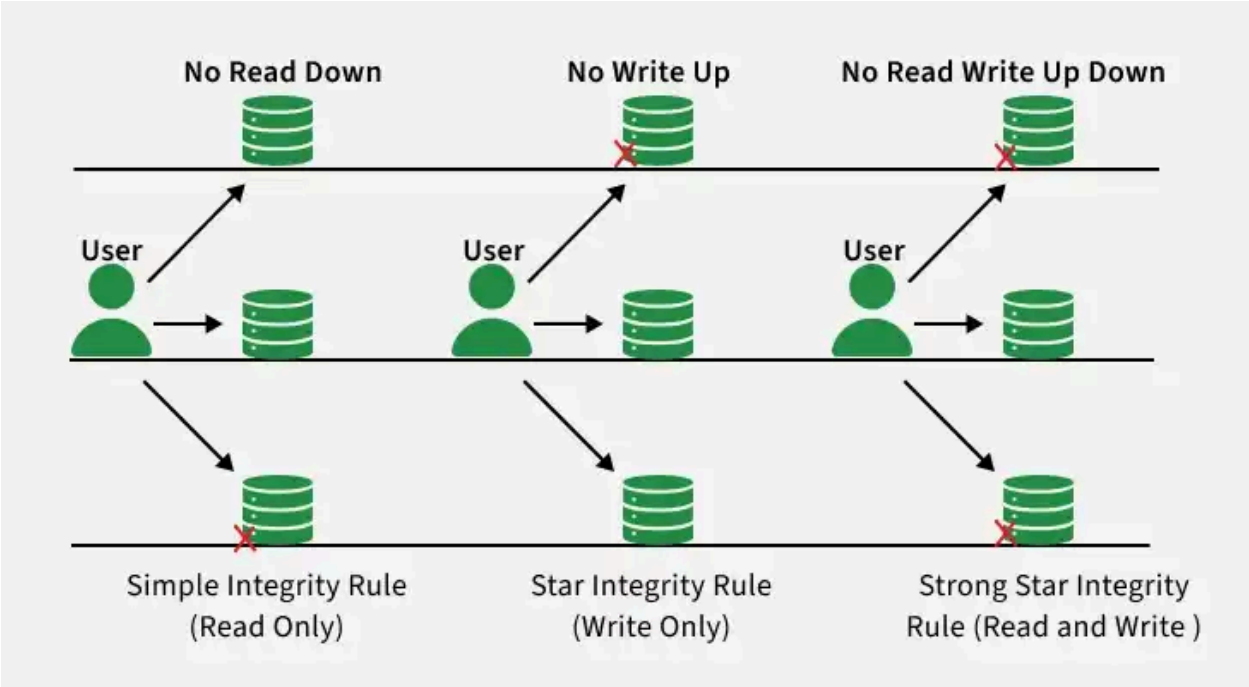

Fig. 02: Biba Model

It has mainly three rules:

1. **Simple Confidentiality Rule:** This rule is also known as NO READ-UP. It ensures that a user can only read the files or documents on the same or lower level of secrecy. But it can't read from the upper level.
2. **Star Confidentiality Rule:** This rule is also known as NO WRITE-DOWN. Here a subject can write only on the same level of secrecy or the upper level of secrecy but not into the lower levels.
3. **Strong Confidentiality Rule:** We call this rule as NO READ WRITE UP DOWN. This rule is defined as a subject can only read and write on the same level of secrecy. He can neither read/write on an upper level nor on a lower level.

The Biba model is useful where unauthorized data modification must be prevented. For example:

1. **Financial Systems:** Prevents the low-level banking applications from changing important financial records.
2. **Medical Systems:** An untrusted user can't modify or access a patient's medical information and medical records.
3. **Audit Logs:** Any kind of unauthorized program can't modify security logs.
4. **Trusted Computing Base (TCB):** It ensures that any user-level program can't change or corrupt any process on the kernel level.

Covert Channels: Subverting Formal Models

Thirdly, a covert channel is a communication path that is being used for transferring information secretly. Therefore, a BLP or Biba model can't detect it. It uses shared resources by which a high-level process can communicate with a low-level process. It does the communication secretly. It uses some indirect effects. For example: timing, resource availability, or system behavior.

Types of Covert Channels:

1. **Storage Channels:** Here the information is transmitted via shared storage resource.

   File-Locking Channel: Two processes keep checking the status of a file to read accordingly. A high-level process locks a file, which is represented by "1" and unlocks it, which is represented by 0.

   Resource Usage Channel: A high-level process consumes most of the memory pages to send signal information. And this is an intentional decision. Then a low-level process checks how much space is left and consumes accordingly.

2. **Timing Channels:** Here the information is transmitted through time differences.

   CPU Timing Channel: A high-security process performs heavy CPU work. and sends 1 and then goes to sleep to send 0. A low-level process measures the duration of work to decode the message.

Paging-Rate Channel: A high-level process consumes most of the memory pages and slows down the memory intentionally. Other processes notice the slowdown and try to detect the secret message.

Difficulty of Detection and Prevention: Covert channels are difficult to detect and prevent in real operating systems because they use normal systems activity like resource sharing. It's almost impossible for it to identify normal and malicious activity. Also, complete restriction is not feasible in modern operating systems. If all shared resources are blocked, then the whole system will slow down. Hybrid covert channels make things even worse. Because it make them difficult to

Limitations of Formal Models: Formal models can't ensure a system's whole security. Because it focuses on the explicit access control. They don't focus on things like

1. **Implementation bugs**
2. **Side channels:** Through indirect resources, data leakage can occur.
3. **Human factors:** Data leakage can occur via power usage.

Comparison and Critical Analysis

Comparison Table: Bell-LaPadula vs. Biba Model

Feature	Bell-LaPadula (BLP)	Biba Model
Primary Goal	Confidentiality	Integrity

Main Principle	No Read Up, No Write Down	No Read Down, No Write Up
Read Rule	Read only at or below clearance	Read only at or above integrity
Write Rule	Write only at or above clearance	Write only at or below integrity
Information Flow	Upward toward secrecy	Downward toward integrity
Main Application	Military and classified systems	Financial, medical, and audit systems
Main Weakness	Integrity attacks	Confidentiality leaks

Bell-LaPadula is a confidentiality-oriented model, and Biba is an integrity-oriented model. The BAL model prevents unauthorized users from reading, which keeps the information safe. On the other hand, the Biba model keeps the data accurate by not giving access to modify it to unauthorized users.

In modern operating systems, flexible information sharing is required. So, in modern operating systems, implementing the BLP or Biba model strictly is quite impractical. For example: In Linux or Windows, it needs information sharing for everyday applications. It's not a best practice to use the BLP or Biba model. For example, a user can't copy text from a secret file to an unclassified document. As a solution, adaptations are being used, like Type Enforcement (TE) in SELinux. It is accessed by defined types of subjects and objects, which can interact between them.

Combining Formal Models with Practical Mechanisms

1. **Mandatory Access Control (MAC):** The system sets the rule implementing who can access which part. And no user can change that. It sets the rule-based models like BLP and the Biba model.
2. **Sandboxing:** Like it isolates the program and restricts their access to resources. So that if one application is attacked, the rest of the system is not attacked.
3. **Authentication and Auditing:** Authentication means checking the user before giving access and, while auditing, recording all the activity. So that any suspicious activity can be tracked.
4. **Access Control Lists (ACLs):** A list of rules that decide which user can read, write, or access specific files or documents. And an admin has control over the user's activity.

Conclusion

The Bell-LaPadula and Biba models are foundational to computer security, providing mathematically rigorous methods for enforcing confidentiality and integrity respectively (Rushby, 1995). Bell-LaPadula prevents unauthorized disclosure through "no read up / no write down," while Biba prevents data corruption through "no read down / no write up." Although too rigid for direct use in modern operating systems, their core concepts continue to influence practical security systems like SELinux and Mandatory Access Control frameworks. However, the existence of covert channels proves that formal models alone are not enough for complete security protection. Therefore, effective operating system security requires a defense-in-depth approach that combines formal policies, sandboxing, auditing, secure implementation, and continuous monitoring.

Reference:

1. Bell, D. E., & La Padula, L. J. (1976). Secure computer system: Unified exposition and Multics interpretation. Mitre Corporation.
2. Biba, K. J. (1975). Integrity Considerations for Secure Computer Systems. Mitre Corporation.
3. Okhravi, H., Bak, S., & King, S. T. (2010). Design, Implementation and Evaluation of Covert Channel Attacks. IEEE International Conference on Technologies for Homeland Security.
4. Zanin, G., & Mancini, L. V. (2004). Towards a formal model for security policies specification and validation in the SELinux system. ACM SACMAT Conference.
5. Eaman, A., Sistany, B., & Felty, A. (2017). Review of Existing Analysis Tools for SELinux Security Policies. International Conference on Security and Management.
6. https://www.geeksforgeeks.org/computer-networks/introduction-to-classic-security-models/